République Algérienne Démocratique et Populaire

Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

Université des Sciences et de la Technologie Houari Boumediene

Faculté d'Informatique

Département Sécurité Informatique

Implémentation d'un Système de Communication Sécurisée et Analyse des Algorithmes Cryptographiques

Sécurité Informatique

Membres de l'équipe :

Ghribi Ahlam 212231458708

Année académique : 2024/2025

Table des matières

1 Introduction **3**
1.0.1 Objectifs du Rapport . . . 3

2 Algorithmes de Cryptographie Classique **4**
2.1 Chiffre de César . . . 4
2.2 Substitution Aléatoire . . . 4
2.3 Chiffrement Affine . . . 5
2.4 Chiffrement de Hill . . . 6
2.5 Chiffrement de Playfair . . . 6
2.6 Chiffrement de Vigenère . . . 7
2.7 One-Time Pad (OTP) . . . 8

3 Techniques d'Analyse Cryptographique **9**
3.1 Analyse de Fréquence . . . 9
3.2 Test de Kasiski . . . 9
3.3 Méthode du Mot Probable . . . 10
3.4 Indice de Coïncidence . . . 11

4 Algorithmes de Cryptographie Moderne **12**
4.1 RC4 . . . 12
4.2 AES . . . 14
4.3 Fonctions de Hachage . . . 14
4.3.1 MD5 , SHA-256 et BLAKE3 . . . 14
4.3.2 Analyse Comparative . . . 14
4.4 Signatures Numériques . . . 14
4.4.1 ElGamal . . . 15
4.4.2 DSA . . . 16
4.5 Cryptographie à Courbes Elliptiques (ECC) . . . 16
4.6 Diffie-Hellman . . . 17
4.7 Analyse Comparative . . . 18

5 Communication Sécurisée **19**
5.1 Architecture Hybride RSA/AES-GCM . . . 19
5.1.1 Génération des Clés RSA . . . 19
5.2 Protocole d'Établissement de Connexion . . . 19
5.2.1 Étape 1 : Réception de la Clé Publique du Serveur . . . 20
5.2.2 Étape 2 : Envoi de la Clé Publique du Client . . . 20
5.2.3 Étape 3 : Établissement de la Clé de Session AES . . . 20

5.3 Chiffrement AES-GCM des Communications 21
5.3.1 Processus de Chiffrement 21
5.3.2 Processus de Déchiffrement 21
5.4 Structure des Paquets 22
5.5 Gestion des Erreurs et Sécurité 22
5.5.1 Gestion des Exceptions 22
5.5.2 Propriétés de Sécurité 22
5.6 Analyse de Performance 23
5.6.1 Complexité Temporelle 23
5.6.2 Optimisations Possibles 23
5.7 Résultats d'Exécution 23

6 Conclusion **25**
6.1 Résumé 25
6.2 Perspectives 25

Chapitre 1

Introduction

La cryptographie moderne est la pierre angulaire de la sécurité informatique. Ce rapport combine une implémentation pratique d'un système de communication sécurisée basé sur RSA/AES-GCM avec une analyse approfondie des algorithmes cryptographiques classiques et modernes. L'objectif est de démontrer une compréhension des mécanismes cryptographiques à travers leur implémentation, leur analyse de sécurité et leurs applications pratiques.

1.0.1 Objectifs du Rapport

- Implémenter une gamme d'algorithmes cryptographiques, classiques et modernes.
- Développer un système de communication sécurisée utilisant RSA et AES-GCM.
- Analyser la sécurité et les performances des algorithmes.
- Comparer les approches cryptographiques.
- Fournir des captures d'écran des exécutions.

Chapitre 2

Algorithmes de Cryptographie Classique

2.1 Chiffre de César

Le chiffre de César est une substitution monoalphabétique avec un décalage fixe.

```python
def caesar_cipher(text, shift, decrypt=False):
    alphabet = string.ascii_uppercase
    result = []
    for char in text.upper():
        if char in alphabet:
            idx = alphabet.index(char)
            idx = (idx - shift) % 26 if decrypt else (idx + shift) %
               26
            result.append(alphabet[idx])
        else:
            result.append(char)
    return ''.join(result)
```

Listing 2.1 – Chiffre de César

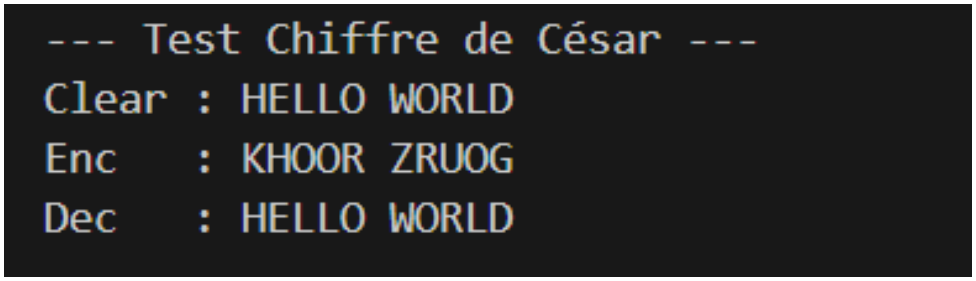

FIGURE 2.1 – Résultat d'exécution du Chiffre de César

2.2 Substitution Aléatoire

La substitution aléatoire remplace chaque lettre par une autre via une permutation aléatoire.

```python
def random_substitution_cipher(text, key=None, decrypt=False):
    alphabet = string.ascii_uppercase
```

```python
    if key is None:
        shuffled = list(alphabet)
        random.shuffle(shuffled)
        key = dict(zip(alphabet, shuffled))
    inv_key = {v: k for k, v in key.items()}
    mapping = inv_key if decrypt else key
    result = [mapping.get(c, c) for c in text.upper()]
    return ''.join(result), key
```

Listing 2.2 – Substitution Aléatoire

```
--- Test Substitution Aléatoire ---
Clair : HELLO WORLD
Enc   : EZPPS CSNPV
Dec   : HELLO WORLD
Clé   : {'A': 'Y', 'B': 'O', 'C': 'Q', 'D': 'V', 'E': 'Z', 'F': 'R', 'G': 'T', 'H': 'E', 'I': 'B', 'J': 'H', 'K': 'M', 'L': 'P', 'M': 'F', 'N': 'L
', 'O': 'S', 'P': 'U', 'Q': 'I', 'R': 'N', 'S': 'X', 'T': 'A', 'U': 'D', 'V': 'W', 'W': 'C', 'X': 'K', 'Y': 'J', 'Z': 'G'}
```

FIGURE 2.2 – Résultat d'exécution de la Substitution Aléatoire

2.3 Chiffrement Affine

Le chiffrement affine utilise une transformation linéaire : $E(x) = (ax + b) \mod 26$.

```python
def hill_cipher(text, key_matrix, decrypt=False):
    assert key_matrix.shape == (2, 2), "Key matrix must be 2x2"
    m = 26
    def mod_inv(mat):
        det = int(round(np.linalg.det(mat)))
        inv_det = pow(det, -1, m)
        adj = np.array([[mat[1,1], -mat[0,1]], [-mat[1,0],
            mat[0,0]]])
        return (inv_det * adj) % m

    km = mod_inv(key_matrix) if decrypt else key_matrix
    text = ''.join(c for c in text.upper() if c in
        string.ascii_uppercase)
    if len(text) % 2:
        text += 'X'

    result = []
    for i in range(0, len(text), 2):
        pair = np.array([[ord(text[i]) - 65], [ord(text[i+1]) - 65]])
        comp = km.dot(pair) % m
        result.extend(chr(int(comp[j,0]) + 65) for j in range(2))
    return ''.join(result)
```

Listing 2.3 – Chiffrement Affine

```
--- Test Chiffrement Affine ---
Enc    : RCLLA OAPLX
Dec    : HELLO WORLD
```

FIGURE 2.3 – Résultat d'exécution du Chiffrement Affine

2.4 Chiffrement de Hill

Le chiffrement de Hill utilise une matrice pour chiffrer des blocs de lettres.

```python
def hill_encrypt(text, key_matrix):
    return ''.join(chr(sum(k * (ord(c) - 97) for k, c in
        zip(key_row, text)) % 26 + 97) for key_row in key_matrix)
```

Listing 2.4 – Chiffrement de Hill

```
--- Test Chiffre de Hill ---
Enc    : HIOZEIPJQL
Dec    : HELLOWORLD
```

FIGURE 2.4 – Résultat d'exécution du Chiffrement de Hill

2.5 Chiffrement de Playfair

Le chiffrement de Playfair utilise une grille 5x5 basée sur une clé.

```python
def playfair_cipher(text, key, decrypt=False):
    # Build 5x5 table without J
    key_str = ''.join(dict.fromkeys((key +
        string.ascii_uppercase).upper().replace('J','')))
    table = [list(key_str[i:i+5]) for i in range(0,25,5)]
    def find_pos(c):
        for r,row in enumerate(table):
            if c in row: return r, row.index(c)

    t = ''.join(c for c in text.upper().replace('J','I') if c in
        string.ascii_uppercase)
    bigrams = []
    i = 0
    while i < len(t):
        a = t[i]
```

```
        b = t[i+1] if i+1 < len(t) else 'X'
        if a == b:
            bigrams.append((a,'X'))
            i += 1
        else:
            bigrams.append((a,b))
            i += 2

    result = []
    for a,b in bigrams:
        ra,ca = find_pos(a)
        rb,cb = find_pos(b)
        if ra == rb:
            shift = -1 if decrypt else 1
            result.append(table[ra][(ca+shift)%5])
            result.append(table[rb][(cb+shift)%5])
        elif ca == cb:
            shift = -1 if decrypt else 1
            result.append(table[(ra+shift)%5][ca])
            result.append(table[(rb+shift)%5][cb])
        else:
            result.append(table[ra][cb])
            result.append(table[rb][ca])
    return ''.join(result)
```

Listing 2.5 – Chiffrement de Playfair

```
--- Test Chiffre de Playfair ---
Enc    : GYIZSCOKCFBU
Dec    : HELXLOWORLDX
```

FIGURE 2.5 – Résultat d'exécution du Chiffrement de Playfair

2.6 Chiffrement de Vigenère

Le chiffrement de Vigenère utilise une clé répétée pour un décalage variable.

```
def vigenere_cipher(text, key, decrypt=False):
    res = []
    key = key.upper()
    ki = 0
    for c in text.upper():
        if c in string.ascii_uppercase:
            shift = ord(key[ki % len(key)]) - 65
            shift = -shift if decrypt else shift
```

```
            res.append(chr((ord(c) - 65 + shift) % 26 + 65))
            ki += 1
        else:
            res.append(c)
    return ''.join(res)
```

Listing 2.6 – Chiffrement de Vigenère

```
--- Test Chiffre de Vigenère ---
Enc    : RIJVS UYVJN
Dec    : HELLO WORLD
```

FIGURE 2.6 – Résultat d'exécution du Chiffrement de Vigenère

2.7 One-Time Pad (OTP)

L'OTP offre une sécurité parfaite avec une clé aléatoire unique.

```

def one_time_pad(text, key):
    assert len(key) >= len(text), "Key too short"
    res = []
    for p,k in zip(text.upper(), key.upper()):
        if p in string.ascii_uppercase:
            res.append(chr(((ord(p)-65) ^ (ord(k)-65)) + 65))
        else:
            res.append(p)
    return ''.join(res)
```

Listing 2.7 – One-Time Pad

```
--- Test Masque Jetable (OTP) ---
Enc    : QIJBF
Dec    : HELLO
```

FIGURE 2.7 – Résultat d'exécution de l'OTP

Chapitre 3

Techniques d'Analyse Cryptographique

3.1 Analyse de Fréquence

L'analyse de fréquence identifie les lettres les plus fréquentes pour casser les chiffrements monoalphabétiques.

```python
def frequency_analysis(text):
    letters = [c for c in text.upper() if c in
       string.ascii_uppercase]
    freq = Counter(letters)
    total = sum(freq.values())
    return {ch: count/total for ch, count in freq.items()}
```

Listing 3.1 – Analyse de Fréquence

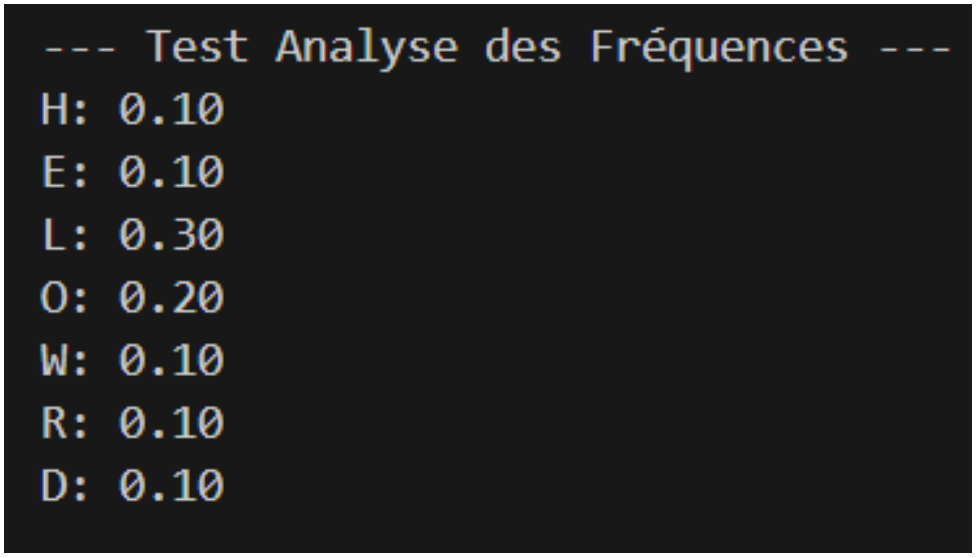

FIGURE 3.1 – Résultat d'exécution de l'Analyse de Fréquence

3.2 Test de Kasiski

Le test de Kasiski identifie la longueur de la clé dans les chiffrements polyalphabétiques.

```python
def kasiski_test(text, n=3):
    t = ''.join(c for c in text.upper() if c in
       string.ascii_uppercase)
    distances = []
    for i in range(len(t)-n+1):
        seq = t[i:i+n]
        for j in range(i+1, len(t)-n+1):
            if t[j:j+n] == seq:
                distances.append(j-i)
    from math import gcd
    gcds = [gcd(a,b) for a in distances for b in distances if a < b]
    return Counter(gcds)
```

Listing 3.2 – Test de Kasiski

```
--- Test Kasiski ---
Counter({3: 39})
```

FIGURE 3.2 – Résultat d'exécution du Test de Kasiski

3.3 Méthode du Mot Probable

Cette méthode recherche des mots probables dans le texte chiffré.

```python
def probable_word_method(ciphertext, word):
    res = []
    for i in range(len(ciphertext)-len(word)+1):
        seg = ciphertext[i:i+len(word)]
        shifts = [(ord(s)-ord(w)) % 26 for s,w in zip(seg.upper(),
           word.upper())]
        key_stream = ''.join(chr(s+65) for s in shifts)
        res.append((i, key_stream))
    return res
```

Listing 3.3 – Méthode du Mot Probable

```
--- Test Méthode du Mot Probable ---
[(0, 'HEL'), (1, 'YFX'), (2, 'ZRU'), (3, 'LOV'), (4, 'IPW'), (5, 'JQA'), (6, 'KUX'), (7, 'ORL'), (8, 'LFP')]
```

FIGURE 3.3 – Résultat d'exécution de la Méthode du Mot Probable

3.4 Indice de Coïncidence

L'indice de coïncidence mesure la probabilité que deux lettres soient identiques.

```python
def index_of_coincidence(text):
    t = [c for c in text.upper() if c in string.ascii_uppercase]
    N = len(t)
    if N < 2: return 0
    freqs = Counter(t)
    return sum(f*(f-1) for f in freqs.values()) / (N*(N-1))
```

Listing 3.4 – Indice de Coïncidence

```
--- Test Indice de Coïncidence ---
Indice de coïncidence: 0.0889
```

FIGURE 3.4 – Résultat d'exécution de l'Indice de Coïncidence

Chapitre 4

Algorithmes de Cryptographie Moderne

4.1 RC4

RC4 est un chiffrement à flux basé sur une permutation pseudo-aléatoire.

```
Clé ASCII     : [109, 97, 95, 99, 108, 101, 95, 115, 101, 99, 114, 101, 116, 101]

--- Début chiffrement RC4 ---
Texte clair    : Bonjour, ceci est un test.

Index | P(char) | K(byte) | C(char)
-----------------------------------
    0 |      66 |      38 |     100 (d)
    1 |     111 |      57 |      86 (V)
    2 |     110 |     158 |     240 (ð)
    3 |     106 |       9 |      99 (c)
    4 |     111 |     101 |      10 (
)
    5 |     117 |      20 |      97 (a)
    6 |     114 |     212 |     166 (¦)
    7 |      44 |     178 |     158 ()
    9 |      99 |     134 |     229 (å)
   10 |     101 |      78 |      43 (+)
   11 |      99 |     209 |     178 (²)
   12 |     105 |      93 |      52 (4)
   13 |      32 |     225 |     193 (Á)
   14 |     101 |      58 |      95 (_)
   15 |     115 |     223 |     172 (¬)
   16 |     116 |     216 |     172 (¬)
   17 |      32 |     179 |     147 ()
   18 |     117 |     119 |       2 (☻)
   19 |     110 |      11 |     101 (e)
   20 |      32 |     197 |     229 (å)
   21 |     116 |     123 |      15 (¤)
   22 |     101 |     200 |     173 ( )
   23 |     115 |     155 |     232 (è)
   24 |     116 |     201 |     189 (¾)
   25 |      46 |     218 |     244 (ô)

Texte chiffré  : dVðc
a¦å+²4Á_¬¬☻eåø è¾ô
```

FIGURE 4.1 – Résultat d'exécution de RC4

```
Texte chiffré  : dVõc
a¦å+²4Á_¬¬®eå¤ è%ô

--- Début déchiffrement RC4 ---
Clé ASCII     : [109, 97, 95, 99, 108, 101, 95, 115, 101, 99, 114, 101, 116, 101]

--- Début chiffrement RC4 ---
Texte clair    : dVõc
a¦å+²4Á_¬¬®eå¤ è%ô

Index | P(char) | K(byte) | C(char)
----------------------------------
    0 |     100 |      38 |      66 (B)
    1 |      86 |      57 |     111 (o)
    2 |     240 |     158 |     110 (n)
    3 |      99 |       9 |     106 (j)
    4 |      10 |     101 |     111 (o)
    5 |      97 |      20 |     117 (u)
    6 |     166 |     212 |     114 (r)
    7 |     158 |     178 |      44 (,)
    8 |     129 |     161 |      32 ( )
    9 |     229 |     134 |      99 (c)
   10 |      43 |      78 |     101 (e)
   11 |     178 |     209 |      99 (c)
   12 |      52 |      93 |     105 (i)
   13 |     193 |     225 |      32 ( )
   14 |      95 |      58 |     101 (e)
   15 |     172 |     223 |     115 (s)
   16 |     172 |     216 |     116 (t)
   17 |     147 |     179 |      32 ( )
   18 |       2 |     119 |     117 (u)
   19 |     101 |      11 |     110 (n)
   20 |     229 |     197 |      32 ( )
   21 |      15 |     123 |     116 (t)
   22 |     173 |     200 |     101 (e)
   23 |     232 |     155 |     115 (s)
```

FIGURE 4.2 – Résultat d'exécution de RC4

```
--- Début chiffrement RC4 ---
Texte clair    : dVõc
a¦å+²4Á_¬¬®eå¤ è%ô

Index | P(char) | K(byte) | C(char)
----------------------------------
    0 |     100 |      38 |      66 (B)
    1 |      86 |      57 |     111 (o)
    2 |     240 |     158 |     110 (n)
    3 |      99 |       9 |     106 (j)
    4 |      10 |     101 |     111 (o)
    5 |      97 |      20 |     117 (u)
    6 |     166 |     212 |     114 (r)
    7 |     158 |     178 |      44 (,)
    8 |     129 |     161 |      32 ( )
    9 |     229 |     134 |      99 (c)
   10 |      43 |      78 |     101 (e)
   11 |     178 |     209 |      99 (c)
   12 |      52 |      93 |     105 (i)
   13 |     193 |     225 |      32 ( )
   14 |      95 |      58 |     101 (e)
   15 |     172 |     223 |     115 (s)
   16 |     172 |     216 |     116 (t)
   17 |     147 |     179 |      32 ( )
   18 |       2 |     119 |     117 (u)
   19 |     101 |      11 |     110 (n)
   20 |     229 |     197 |      32 ( )
   21 |      15 |     123 |     116 (t)
   22 |     173 |     200 |     101 (e)
   23 |     232 |     155 |     115 (s)
   24 |     189 |     201 |     116 (t)
   25 |     244 |     218 |      46 (.)

Texte chiffré  : Bonjour, ceci est un test.

Texte déchiffré : Bonjour, ceci est un test.
```

FIGURE 4.3 – Résultat d'exécution de RC4

4.2 AES

AES est un standard de chiffrement symétrique à blocs.

```
C:\Users\may\Downloads\algo\aes>python test_aes.py
=== Test d'encryption AES en mode CBC ===
Message original: Hello, AES! This is a test message.
Taille du message: 35 bytes

Clé AES (hex): 803d62f5a7bf91acb33f048041b48414
IV (hex): 5e317683bf33374c5cc4db439b997b39

Message chiffré (hex): f93302f99609a2626e87645b6f74472031f0c2373c8f463daed478b689d0d1f2cba1ef7339233beb86b464c3cc06996d

Message déchiffré: Hello, AES! This is a test message.

✅ Test réussi: Le message déchiffré correspond au message original!
```

FIGURE 4.4 – Résultat d'exécution de AES

4.3 Fonctions de Hachage

Les fonctions de hachage produisent un condensé avec résistance à la préimage, à la seconde préimage et aux collisions.

4.3.1 MD5 , SHA-256 et BLAKE3

MD-5:

```
PS C:\Users\lenovo\OneDrive\Bureau\suite_crypto> python -u "c:\Users\lenovo\OneDrive\
Bureau\suite_crypto\md5.py"
7480b85c502d8d8582cace61544b730f
```

SHA-256:

```
Hash SHA-256 : 67ab7af89d26770902d60ea2d1b6f50fd0e8ca79d48133c5fd90fe02de5817bc
```

BLAKE3:

```
Hash BLAKE3 : d968f2a0a648fe7e436e072377a0271c4ae144e223a2b607139d142803344a85
```

FIGURE 4.5 – Résultat d'exécution de l'implémentation MD5 , SHA-256 et BLAKE3

4.3.2 Analyse Comparative

Algorithme	Taille (bits)	Sécurité	Performance	Usage
MD5	128	Faible	Rapide	Obsolète
SHA-256	256	Élevée	Modérée	Standard actuel
BLAKE3	256	Très élevée	Très rapide	Moderne

TABLE 4.1 – Comparaison des fonctions de hachage

4.4 Signatures Numériques

Les signatures numériques garantissent l'authenticité et l'intégrité.

```
Génération des clés...
Clé privée : -----BEGIN RSA PRIVATE KEY-----
MIIEowIBAAKCAQEAyfHFBPtUHK7S7w8nH44p6CBAS1sk9+M+SFu3XCiidv95xUbQ
4uoAJJPzW4D6TzEHqCH9MflX0AVDzezDZoYDARI8WAxuKk7ApDchkAm58jvjubsb
g/uznJE2VmkLIgHEN0HLLEBiRdUqAYGJ7SP67oJ1qN98V2fj4wy9ucj23oaZ3cG6
h+nQHPwWheKpvWZ4eByiFoHx5aPA1TVm8l80nyT6PesJ/rhsoiewTLVxh0Be4Q3b
Fx+IuMCIYkDbdKSO1ausLGwt7RlvawOF23cfAByUNaN97g/1sRHPQ0hX5CMFuDPY
fR2HucCROlempybniNeiAR20g1Q5gwAJwyhMwQIDAQABAoIBACDvDPKDCnSjDkNu
dTLwPGVe3R1s/9ptdPrX+k9NUTGW+ITzx506DEvq1QVXQ3smw9jnsiM0Rt3l/ejX
dAaEVHkYQD8bx9uefExLsW2MKIcYvUrM+O/NXL1r8TCmcOpa32Ujlb7hwSUiXndg
aN4DwCilngqsZ9rZVpDwwwBJhflEMCCxGAb3WKigD0ezv4HSRxafkVvM9mwBLaDQ
YvrEESQ5r3ClOj9SXlKq/awdeYa2F0syvDhDc4QXdg0Uw8dr2orGtW31WusBZpd2
RNwYbHwPt+1z6CQWJQvKLoduzR0vb3xl8bX4SKu01gJKH0zEnwX/pcXADXzDrH3C
89CGVAECgYEA3EipJxeIpyaJKael20CQ6qmjrpfpcJFxecDwh+hVTutKK2Df/5go
lOOu/Eh2Hs/FasQ1MrDpg2Ph8l9QW5pWAkBrF2b4ZV70lq8pYs7vvJO3VEoQ8sc8
9m57qMzffc6U+RyrL9yKnryko9uDAx8h/uT4ph7JSlxgjiVh+NyfzQUCgYEA6q/k
oWzyCAFal5q3RMX9MmI9vfPnyAMm7uNDP1jQ5LzLAfgkErkJxHYzVGvWLpeuWsnT
4Z7v/Xx2fJv5Sw7RkWgAMWbrI8GV3Lyxmi2NT4mcuRb6RA/fCrX1VAdV9uw/kOxE
Gu2/ZdPHkyPPPEbGiuB0Kb4qkI0/U/Q91i64rY0CgYEAwmK8Ier6g8XVtBRkn53S
Q8F+10rmjJdL84z2tHepEANBaBYMlJfI99oTSD0JPxdJtnV+HEVRSsiRUjgZyTvg
OgxudDBBcfLhEzdu5eqGU3pmhNL3McXvXecQ6Fwhu/rYKxvJVGrhSEJfZtkGO3L1
pOCZKr5wElaCPe1vDxRhT6ECgYA1VrrA0ZEErsIXOMu0bsdIHznnJTslo0nC+5X6
HMaFZj9BeFNCihmlNDQgWM36EvFGox5r5gKnexU25wA2xextftSkZb7Fhqb3308C
Op4mWKFtE8LXmz/XIFncujoYf8NGvdwQvr5Ed2ihkX71NSTExZTBM/5aki+1dgpj
1oSUtQKBgFFw08RSUajF5j8tQh+ZwWtgMbB+T9TkLaCLtWiuwE5igulE7cBrT/1b
8iaDcft70KMRJaQgcAY5KrckZwFB3sRfT0mZjB9ouBptj7Yvks0P5JxmelP+PuNL
56mVpVtQJrD0wwqJ2cIs6NQNFY5TXj+wQ50F8rD1WwPkvNqcfjcP
-----END RSA PRIVATE KEY-----
```

FIGURE 4.6 – Résultat d'exécution de la signature RSA

4.4.1 ElGamal

```
Génération des clés...

Clé publique :
p = 58499683085733288640254158989394599879522046941890683507321398106525740798919
g = 40155462216607934144458420509858487364414786425844019851218809269711205412301
h = 29679254787267152501337547291908084434919067975307374254392929081538135243739
Nombre de bits = 256

Clé privée :
p = 58499683085733288640254158989394599879522046941890683507321398106525740798919
g = 40155462216607934144458420509858487364414786425844019851218809269711205412301
x = 32598762196875644416160682596538753856256052965015911062062865183687532112 6
Nombre de bits = 256

Message original : cyber_security

Chiffrement du message...
Message chiffré : 28755695828335362402705192986315004499928582811844537077801646013564181014 96 25186113881269420820325829967774288749015117890803404254794104573195204398 41

Déchiffrement du message...
Message déchiffré : cyber_security
```

FIGURE 4.7 – Résultat d'exécution de l'implémentation ElGamal

4.4.2 DSA

```
Génération des paramètres DSA...
Génération de q...
Génération de p...
Génération de g...
Paramètres générés:
p (1024 bits): 0xe2790c5d763c637252e2435c768b8bf11f92173b7d4d53e2bbd233d50b82b496ec79ec0bfd9433d5f98ae2435ef72d15bdc48bb86e9173612a87cb0d4685c0b0
120c19d7a0abe191a591df5e5fa491637c5bef1bdab399287f03811f13ee5a86cc8ef3196d1e4a504889af5786235701c025120590c9b374e90ed8b1606a28b
q (160 bits): 0xf3821cfdc083b73a473bd358610e6a64e1301617
g: 0x6c4d98458a0175254211bc6120ab4c121167862187bea936875bc685939443c12d412522f086ae0fbf9d25c79b6ecf71beebf9f89d494a7d38ae77b2a36b9fb5170a59dcee66
d8218113a7e3dcf1a6e668aa2deafb98a96eb0e438517e76167d6e51021e3ed10acf4cd600882db2359a2f3f58239aaf371190f8247be334b7d

Clés générées:
Clé privée (x): 0x13335bc9bb635ff529310971601e0211e34f76b5
Clé publique (y): 0x248d1b432c8fb13680c523726622a8175b20f7c8008b07c93776f1460aa0f77022fabe8eb6f2527448d46b30ee0a5d029cfc317b7daaf44dff9e293886ab6
a16598183aadacf8af2c7e82446a4f34457dd509b98f02f874b59ce29a7cd7f4e0a5a4db937d81400a64a84a89a5d562ebcef444c150eea184705fa014cf2149b8

Message à signer: Bonjour, ceci est un message secret!

Signature en cours...
Signature générée:
r: 0xab484bf8b299b988604822cb7b99d8f3a00e3f3e
s: 0xc54107fb844ca507c8d2e3822c48e370b510a5cc

Vérification de la signature...
Signature valide: True

Test avec un message modifié...
Signature valide pour le message modifié: False

Test avec une signature modifiée...
Signature modifiée valide: False
```

FIGURE 4.8 – Résultat d'exécution de l'implémentation DSA

4.5 Cryptographie à Courbes Elliptiques (ECC)

Une courbe elliptique est définie par $y^2 \equiv x^3 + ax + b \pmod{p}$.

```
Menu:
1. Vérifier la validité de la courbe
2. Vérifier si un point est sur la courbe
3. Échange de clé Diffie-Hellman (ECC)
4. Quitter
Choix (1-4) : 1
Courbe valide ? True

Menu:
1. Vérifier la validité de la courbe
2. Vérifier si un point est sur la courbe
3. Échange de clé Diffie-Hellman (ECC)
4. Quitter
Choix (1-4) : 3
Clé privée A : 09
Clé privée B : 33
Clé publique A : (7, 6)
Clé publique B : (9, 1)
Clé partagée (A) : (0, 11)
Clé partagée (B) : (0, 11)
Clé identique ? True

Menu:
1. Vérifier la validité de la courbe
2. Vérifier si un point est sur la courbe
3. Échange de clé Diffie-Hellman (ECC)
4. Quitter
Choix (1-4) :
```

FIGURE 4.9 – Résultat d'exécution de l'implémentation ECC

4.6 Diffie-Hellman

```
C:\Users\may\Downloads\rsa>python diffie.py
Sadat envoie la clé partielle : 147
Michael envoie la clé partielle : 66
Clé complète chez Sadat   : 75
Clé complète chez Michael : 75
Message chiffré : ³´¾k´¾k¬kÁ°½Äk¾°®½°¿k¸°¾¾¬²°lll
Message déchiffré : This is a very secret message!!!
```

FIGURE 4.10 – Résultat d'exécution de Diffie-Hellman

4.7 Analyse Comparative

Algorithme	Vitesse	Sécurité	Usage Recommandé
MD5	Très rapide	Faible	Vérification d'intégrité
SHA-256	Rapide	Élevée	Applications générales
BLAKE3	Très rapide	Très élevée	Haute performance
RSA-2048	Lent	Élevée	Signatures, échange de clés
ECC-256	Rapide	Très élevée	Applications mobiles, IoT
3DES	Lent	Modérée	Systèmes legacy
AES	Rapide	Élevée	Chiffrement standard
RC4	Rapide	Faible	Obsolète

TABLE 4.2 – Analyse comparative des algorithmes

Niveau de sécurité	RSA (bits)	ECC (bits)
80 bits	1024	160
112 bits	2048	224
128 bits	3072	256
192 bits	7680	384
256 bits	15360	512

TABLE 4.3 – Comparaison des tailles de clés RSA vs ECC

Algorithme	Vulnérabilité	Contre-mesure
MD5	Collisions	Utiliser SHA-256+
RSA	Factorisation	Clés 2048+ bits
DES	Clé courte	Utiliser AES
ECC	Courbes faibles	Courbes standardisées
DSA	k réutilisé	Nouveau $-sys$ par signature
RC4	Biais dans le flux	Éviter son usage

TABLE 4.4 – Vulnérabilités et contre-mesures

Chapitre 5

Communication Sécurisée

5.1 Architecture Hybride RSA/AES-GCM

Le système implémente une architecture de chiffrement hybride combinant RSA pour l'échange de clés et AES-GCM pour le chiffrement des données. Cette approche tire parti de la sécurité du RSA pour établir un canal sécurisé et de l'efficacité de l'AES pour les communications en temps réel.

5.1.1 Génération des Clés RSA

Chaque composant (serveur et client) génère sa propre paire de clés RSA lors de l'initialisation :

```
# G n ration des cl s RSA c t  client
private_key = rsa.generate_private_key(public_exponent=65537,
   key_size=2048)
public_key = private_key.public_key()

# S rialisation de la cl  publique pour transmission
public_pem = public_key.public_bytes(
    encoding=serialization.Encoding.PEM,
    format=serialization.PublicFormat.SubjectPublicKeyInfo
)
```

Listing 5.1 – Génération des clés RSA côté client

Paramètres choisis :

- **Exposant public** : 65537 (standard recommandé)
- **Taille de clé** : 2048 bits (sécurité équivalente à AES-112)

5.2 Protocole d'Établissement de Connexion

Le protocole suit un handshake en trois étapes garantissant l'authentification mutuelle et l'établissement d'une clé de session partagée.

5.2.1 Étape 1 : Réception de la Clé Publique du Serveur

```
def start_client():
    client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    client.connect(('172.20.10.2', 5555))
    print("Connect  au serveur")

    # R ception de la cl  publique du serveur
    server_public_pem = client.recv(2048)
    server_public_key =
       serialization.load_pem_public_key(server_public_pem)
    print("Cl  publique du serveur re ue")
```

Listing 5.2 – Établissement de la connexion client

5.2.2 Étape 2 : Envoi de la Clé Publique du Client

```
    # Envoi de la cl  publique du client
    client.send(public_pem)
    print("Cl  publique du client envoy e")
```

Listing 5.3 – Échange de clés publiques

5.2.3 Étape 3 : Établissement de la Clé de Session AES

Le client génère une clé AES-256 aléatoire, la chiffre avec la clé publique du serveur et l'envoie :

```
    # G n ration d'une cl  AES-256 (32 bytes)
    session_key = os.urandom(32)

    # Chiffrement RSA-OAEP de la cl  de session
    encrypted_session_key = server_public_key.encrypt(
        session_key,
        padding.OAEP(
            mgf=padding.MGF1(algorithm=hashes.SHA256()),
            algorithm=hashes.SHA256(),
            label=None
        )
    )
    client.send(encrypted_session_key)
    print("Cl  de session AES envoy e")
```

Listing 5.4 – Génération et envoi de la clé de session AES

Sécurité du protocole OAEP :

- **MGF1** : Fonction de génération de masque basée sur SHA-256
- **Hash** : SHA-256 pour l'intégrité
- **Label** : Paramètre optionnel (None dans notre implémentation)

5.3 Chiffrement AES-GCM des Communications

Une fois la clé de session établie, toutes les communications utilisent AES-GCM pour garantir confidentialité et intégrité.

5.3.1 Processus de Chiffrement

```python
while True:
    message = input("Entrez votre message (ou 'quit' pour quitter): ")
    if message.lower() == 'quit':
        break

    message_bytes = message.encode('utf-8')
    nonce = os.urandom(16)  # IV al atoire de 128 bits

    # Cr ation du cipher AES-GCM
    cipher = Cipher(algorithms.AES(session_key), modes.GCM(nonce))
    encryptor = cipher.encryptor()
    ciphertext = encryptor.update(message_bytes) + encryptor.finalize()

    # Structure du paquet : nonce (16) + tag (16) + ciphertext
    data_to_send = nonce + encryptor.tag + ciphertext
    client.send(data_to_send)
```

Listing 5.5 – Chiffrement des messages avec AES-GCM

5.3.2 Processus de Déchiffrement

```python
    # R ception des donn es chiffr es
    data = client.recv(2048)
    if not data:
        print("Connexion ferm e par le serveur")
        break

    # Extraction des composants
    received_nonce = data[:16]      # Nonce (16 bytes)
    received_tag = data[16:32]      # Tag d'authentification (16 bytes)
    received_ciphertext = data[32:] # Donn es chiffr es

    # D chiffrement et v rification d'int grit
    cipher = Cipher(algorithms.AES(session_key), modes.GCM(received_nonce, received_tag))
    decryptor = cipher.decryptor()
    plaintext = decryptor.update(received_ciphertext) + decryptor.finalize()
    print(f"Message re u du serveur: {plaintext.decode('utf-8')}")
```

Listing 5.6 – Déchiffrement des messages reçus

5.4 Structure des Paquets

Chaque message chiffré suit cette structure fixe :

Composant	Taille	Description
Nonce	16 bytes	Vecteur d'initialisation unique
Tag	16 bytes	Tag d'authentification GCM
Ciphertext	Variable	Données chiffrées

TABLE 5.1 – Structure des paquets de données chiffrées

5.5 Gestion des Erreurs et Sécurité

5.5.1 Gestion des Exceptions

Le code implémente une gestion robuste des erreurs à plusieurs niveaux :

```
try:
    # Code de communication
except Exception as e:
    print(f"Erreur lors de l' change  de messages: {e}")
    break
finally:
    try:
        client.close()
    except:
        pass
    print("Client ferm ")
```

Listing 5.7 – Gestion des erreurs

5.5.2 Propriétés de Sécurité

Confidentialité :

— RSA-2048 pour l'échange de clés (sécurité asymétrique)
— AES-256-GCM pour les données (sécurité symétrique)

Intégrité :

— RSA-OAEP empêche les attaques par malléabilité
— GCM fournit l'authentification intégrée

Authentification :

— Échange mutuel des clés publiques
— Vérification implicite par le succès du déchiffrement

Non-répudiation :

— Possibilité d'ajouter des signatures numériques RSA

5.6 Analyse de Performance

5.6.1 Complexité Temporelle

— **Handshake initial** : O(1) - 3 opérations RSA
— **Chiffrement par message** : O(n) - où n est la taille du message
— **Latence réseau** : Dominante dans les communications réelles

5.6.2 Optimisations Possibles

— Mise en cache des objets Cipher pour éviter la réinitialisation
— Utilisation de threads pour la réception asynchrone
— Compression des données avant chiffrement pour les gros messages
— Implémentation d'un pool de connexions pour les applications multi-clients

5.7 Résultats d'Exécution

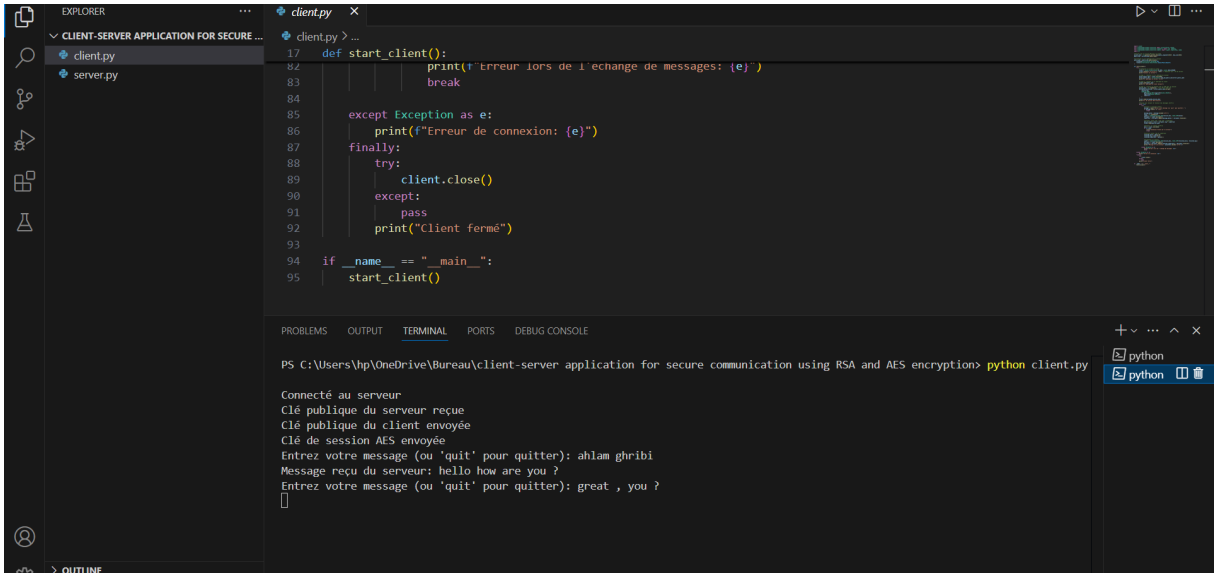

FIGURE 5.1 – Exécution du client avec échange de messages sécurisés

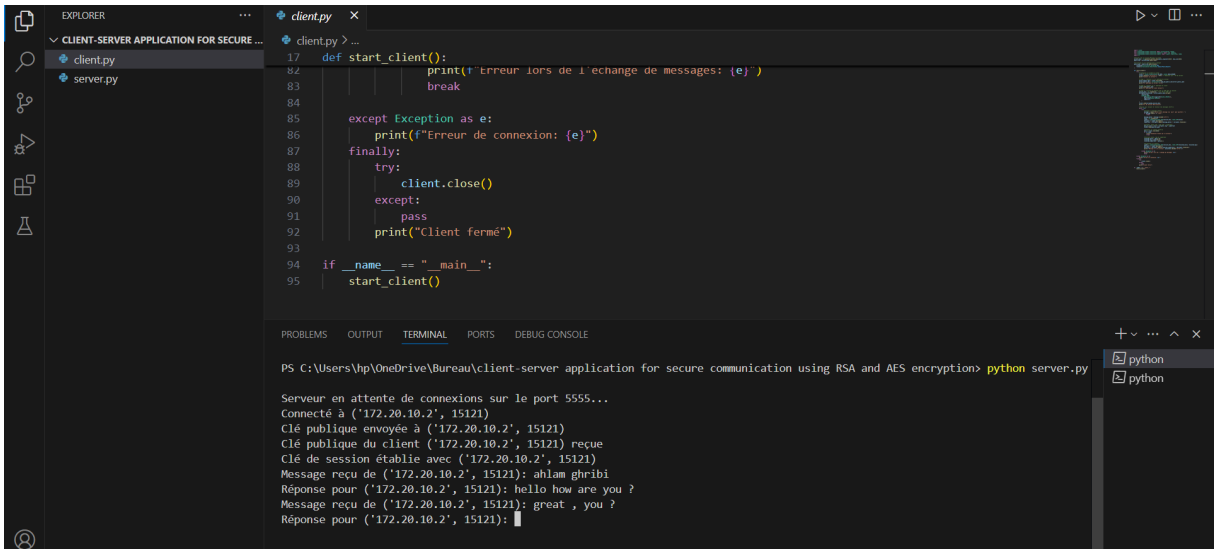

FIGURE 5.2 – Exécution du serveur avec gestion multi-clients

Chapitre 6

Conclusion

6.1 Résumé

Ce rapport a présenté une implémentation complète d'un système de communication sécurisée basé sur RSA/AES-GCM, ainsi qu'une analyse détaillée des algorithmes cryptographiques classiques et modernes. Les points clés incluent :

- L'implémentation d'algorithmes classiques (César, Vigenère, etc.) et leurs vulnérabilités.
- L'évolution des fonctions de hachage (MD5 à BLAKE3).
- La supériorité d'ECC sur RSA pour les applications modernes.
- L'efficacité du chiffrement hybride RSA/AES-GCM pour la communication sécurisée.

6.2 Perspectives

- Exploration de la cryptographie post-quantique.
- Optimisation pour l'IoT et les appareils à faible puissance.
- Étude des techniques de cryptographie homomorphe et des preuves à divulgation nulle.